

Práctica Final: Sistema de gestión de matrículas universitarias

Dpto. Ciencias de la Computación e Inteligencia Artificial
E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada



Estructuras de Datos

Grado en Ingeniería Informática
Doble Grado en Ingeniería Informática y ADE
Doble Grado en Ingeniería Informática y Matemáticas

1. Introducción

En esta práctica final se pretende resolver un problema real eligiendo las mejores estructuras de datos para las operaciones que se piden

Los requisitos para poder realizar esta práctica son:

1. Haber estudiado el Tema 1: Introducción a la eficiencia de los algoritmos
2. Haber estudiado el Tema 2: Abstracción de datos. Templates.
3. Haber estudiado el Tema 3: T.D.A. Lineales.
4. Haber estudiado el Tema 4: STL (especialmente los tipos <set> y <map> e Iteradores.
5. Haber estudiado el tema 5: Árboles

2. Objetivo.

El objetivo de esta práctica es llevar a cabo el análisis, diseño e implementación de un proyecto. Con tal fin el estudiante abordará un problema donde se requiere estructuras de datos que permiten almacenar grandes volúmenes de datos y poder acceder a ellos de la forma más eficiente. Se busca incentivar el trabajo en equipo y la coordinación entre sus miembros, así como integrar de forma responsable el uso de LLM's en su elaboración. La práctica tiene un formato abierto para que cada grupo aporte a su diseño todo aquello que se les ocurra para mejorar su funcionalidad. Habrá por tanto una parte básica obligatoria y otra opcional que necesariamente habrá de ser diferente en cada grupo.

2.1. Tareas a realizar

Se desea implementar un sistema de gestión de matrículas universitarias. El sistema permitirá administrar: Estudiantes, Asignaturas y Matrículas de estudiantes en asignaturas. Se deberá garantizar la no duplicidad de ciertos elementos y permitir búsquedas eficientes. Cualquier estructura de datos que se use deberá estar debidamente justificada.

El sistema deberá permitir:

1. Dar de alta y de baja estudiantes.
2. Crear y eliminar asignaturas.
3. Matricular a un estudiante en una asignatura.
4. Desmatricular a un estudiante.
5. Listar estudiantes ordenados por su identificador.
6. Listar asignaturas existentes.
7. Consultar las asignaturas en las que está matriculado un estudiante.
8. Otras funcionalidades añadidas por el diseñador

Se puede y debe usar la STL en todos los módulos

2.1.1. Restricciones del sistema

Se tendrán en cuenta las siguientes restricciones:

1. Cada estudiante estará identificado por un ID único (entero o cadena).
2. Las asignaturas no pueden duplicarse.
3. Un estudiante puede estar matriculado en varias asignaturas.
4. Una asignatura puede tener múltiples estudiantes.
5. El sistema debe evitar inconsistencias (por ejemplo, matrículas duplicadas).

2.1.2. Estructuras de datos obligatorias

1.- Uso obligatorio del **tipo <set>**. Se deberá utilizar set (al menos) en los siguientes casos:

- Almacenamiento de asignaturas (sin duplicados).
- Almacenamiento de conjuntos de estudiantes matriculados en una asignatura.
- Cualquier otro conjunto donde la unicidad y el orden sean relevantes.

El estudiante deberá definir correctamente el operador < cuando sea necesario y justificar el criterio de ordenación utilizado.

2.- Uso obligatorio del **tipo <map>**. Se deberá utilizar map para (al menos):

- Asociar el ID del estudiante con su información.
- Asociar una asignatura con el conjunto de estudiantes matriculados.
- Gestionar búsquedas eficientes por clave.

Será obligatorio manejar correctamente los casos en los que una clave no exista en el mapa.

3. En el diseño del sistema será **obligatorio** usar al menos las siguientes clases:

- Clase Estudiante
- Clase Asignatura
- Clase SistemaMatriculas

Se deberá justificar el uso de cada estructura de datos en el diseño final.

4. Respecto a la **implementación** deberá incluir:

- Inicialización correcta de los contenedores.
- Un menú interactivo por consola.
- Control de errores en la entrada de datos.
- Código modular y legible.
- Comentarios explicativos de cada módulo.
- Documentación completa del código (usando doxygen u otra herramienta)

5. **Análisis de eficiencia**. El estudiante deberá analizar:

- Eficiencia de inserción y búsqueda en set.
- Eficiencia de inserción y búsqueda en map.
- Impacto del uso de estas estructuras en la escalabilidad del sistema.

6. **Pruebas.** Se deberán describir y probar casos como:

- Matricular un estudiante ya matriculado.
- Eliminar un estudiante con matrículas activas.
- Consultar información inexistente.
- Listados completos y ordenados.

7.- **Ficheros.** Se deberán crear ficheros ficticios de estudiantes y asignaturas ajustándolos a las necesidades del sistema

2.1.3. Estructuras de datos opcionales

Se deberá pensar en cómo podría mejorarse la funcionalidad del sistema añadiendo nuevos módulos que cada grupo de trabajo debe pensar y justificar: Estadísticas de matrículas, Gestión de expedientes...El reto en este apartado es que deben usarse otras estructuras de datos diferentes al <set>/<map> de la parte obligatoria: p.ej: <unordered_map>, <vector>, <list>, <deque>, <queue>...Cuantas más estructuras se usen y mejor justificadas estén para los módulos nuevos en que se piense, mejor será la nota en la parte opcional.

3. TDA y Programas

A modo de ejemplo de clases y pequeño programa de prueba se proporciona esto simplemente como referencia:

```
#include <iostream>
#include <string>
#include <map>
#include <set>
using namespace std;


/* ============================
   Clase Estudiante
   ============================ */
class Estudiante {
private:
    int id;
    string nombre;
public:
    Estudiante(int id, const string& nombre)
        : id(id), nombre(nombre) {}
```

```
    int getId() const {
        return id;
    }
    string getNombre() const {
     //Código
    }
    void mostrar() const {
      //Código
    }
};
/* ============================
   Sistema de Matrículas
   ============================ */
class SistemaMatriculas {
private:
    map<int, Estudiante> estudiantes;              // ID -> Estudiante
    set<string> asignaturas;                       // Asignaturas únicas
    map<string, set<int>> matriculas;              // Asignatura -> IDs estudiantes
public:
    /* ----- Gestión de estudiantes ----- */
    void altaEstudiante(int id, const string& nombre) {
        //Código
    }
    void bajaEstudiante(int id) {
          //Código
        // Debería eliminar al estudiante de todas las matrículas
        }
    /* ----- Gestión de asignaturas ----- */
    void crearAsignatura(const string& nombre) {
        //Código
    }
    void eliminarAsignatura(const string& nombre) {
        //Código
    }
    /* ----- Matrículas ----- */
    void matricular(int id, const string& asignatura) {
      //Código
    }
```

```
    void desmatricular(int id, const string& asignatura) {
      //Código
    }
    /* ----- Consultas ----- */
    void listarEstudiantes() const {
       //Código
    }
    void listarAsignaturas() const {
       //Código
    }
    void estudiantesPorAsignatura(const string& asignatura) const {
        //Código
    }
};
/* ============================
   Programa principal
   ============================ */
int main() {
    SistemaMatriculas sistema;
    // Datos de prueba
    sistema.altaEstudiante(1, "Ana");
    sistema.altaEstudiante(2, "Luis");
    sistema.altaEstudiante(3, "María");

    sistema.crearAsignatura("Estructuras de Datos");
    sistema.crearAsignatura("Metodología de la Programación");

    sistema.matricular(1, "Estructuras de Datos");
    sistema.matricular(2, "Estructuras de Datos");
    sistema.matricular(3, "Metodología de la Programación");

    sistema.listarEstudiantes();
    sistema.listarAsignaturas();
    sistema.estudiantesPorAsignatura("Estructuras de Datos");

    return 0;
}
```

4. Práctica a entregar

El estudiante deberá empaquetar todos los archivos relacionados en el proyecto en un archivo con nombre "**practicafinal.*tgz***" y entregarlo antes de la fecha que se publicará en la página web de la asignatura. Tenga en cuenta que no se incluirán ficheros objeto, ni ejecutables. Es recomendable que se haga una "limpieza" para eliminar los archivos temporales o que se puedan generar a partir de los fuentes.

El estudiante debe incluir el archivo *Makefile* para realizar la compilación. Tenga en cuenta que los archivos deben estar distribuidos en directorios *(directorio de fuentes, de datos...)*

Para realizar la entrega, en primer lugar, se debe realizar la limpieza de archivos que no se incluirán en ella, y situarse en el directorio superior (supongamos que la carpeta se denomina "practica__final") de dónde debe colgar todo para ejecutar:

```
prompt% tar zcv practicafinal.tgz practica_final
```

tras lo cual, dispondrá de un nuevo archivo practicafinal.tgz que contiene el directorio principal así como todas las carpetas y archivos que cuelgan de ella.

La práctica se deberá realizar obligatoriamente (salvo excepciones claramente justificadas) por equipos de 3 personas. Todos los miembros del equipo deberán subir los ficheros a prado a través del enlace correspondiente a la entrega de la práctica final. Se deberá incluir un fichero readme.txt en el que se indique exactamente:

- ***Qué se ha hecho y cómo se ha hecho (incluyendo referencias)***
- ***En qué partes de la práctica se ha involucrado cada miembro del equipo***
- ***Si hay algo que no funciona***
- ***Si se ha usado algún LLM y en qué partes exactamente se ha usado, justificándolo con precisión.***

5. Valoración

*Se podrá alcanzar una **máxima puntuación** en la práctica de **1.5 puntos que será la misma para todos los miembros del equipo**. La práctica habrá de ser **defendida por todo el equipo** en una sesión de prácticas específica para ello.*

No se excluye el uso de LLM's,** pero si se detecta el uso de alguno y no está documentado y justificado, la práctica se valorará con un máximo de **0.25 puntos.

*Se valorará con un máximo de **1 punto la parte básica***

*Se valorará con un máximo de **0.5 puntos la parte opcional***

6. Consideración final

Se pasarán programas anti-plagio cruzando programas de todos los grupos de prácticas de todos los grupos del grado y los dobles grados de la asignatura, de forma que se encomienda a los estudiantes a tener un comportamiento ético en la elaboración de esta práctica final. La detección de plagio conllevará una puntuación global de 0 puntos en las prácticas de todos los miembros del equipo independientemente de las demás notas que se tuvieran.

7. Referencias

- Rosa Rodriguez-Sánchez, J. Fdez-Valdivia, J.A. García, Estructuras de Datos en C++. Un enfoque práctico, 2020
- Garrido, A., Fdez-Valdivia, J. "*Abstracción y estructuras de datos en C++*". Delta publicaciones, 2006.
- Garrido, A. Estructuras de datos avanzadas: con soluciones en C++. Editorial Universidad de Granada. 2025